Sparse-Support Event Elimination for Exact VUS Evaluation

Anonymous research artifact

20 October 2026
Working manuscript

Abstract

Volume Under the Surface (VUS) evaluates time-series anomaly scores across both thresholds and tolerance widths. Its threshold-dependent label mass makes a naive replacement by a weighted ROC curve incorrect. We show that thresholds admitting only positions outside the maximum anomaly support can be removed exactly, provided the intervening negative rectangle and subsequent support transition are integrated separately. Sorting only support scores and histogramming all remaining scores gives exact scalar VUS-ROC and VUS-PR in $O(n\log(Q+1)+(L+1)Q)$ time and $O(n+Q)$ memory, where Q counts positions in the maximum expanded anomaly support and L is the largest buffer width. The algorithm preserves a precisely pinned existing metric. We verify equivalence against the unmodified official implementation and evaluate 350 public series with 1050 released foundation-model score vectors. The compiled implementation achieves a median 2.06× speedup over the fastest of three NumPy/Numba dense implementations. The 250-threshold approximation changes 84 of 2,100 VUS-PR model-pair orders, affecting 55 series, and has maximum absolute error 0.583. The result provides a support-sensitive reduction with an explicit exactness proof and makes full-threshold evaluation practical without sampling-induced ranking changes.

1 Introduction

VUS averages range-aware ROC or precision–recall integrals across tolerance widths, accounting for both overlap and whether an anomaly interval is hit [1, 2]. Efficient evaluation matters: ESA-ADB explicitly excluded VUS because of computational cost [8]. That observation motivates speed improvements; it does not establish suitability for ESA-ADB's other evaluation requirements.

The official optimized evaluator partitions static and dynamic labels but still visits sampled thresholds repeatedly [2]. A straightforward dense exact rank sweep avoids repeated threshold scans by scanning the full sequence once per width. Our key observation is that only positions near labeled anomalies can change the weighted positive state. All other score thresholds have an analytically integrable effect.

Exact evaluation also avoids an accuracy limitation of a fixed uniform-rank threshold grid. At width zero, place the only anomaly at score rank two and append increasingly many lower-scored negatives. Exact PR remains $1/2$, whereas a fixed T-threshold grid first detects it at rank $1+\lfloor(n-1)/(T-1)\rfloor$, and its PR tends to zero. Appendix A gives a version whose absolute error approaches one for any fixed width range. This elementary counterexample motivates retaining all thresholds; it is not specific to the sparse evaluator.

Our target is the pair of scalar outputs of `RangeAUC_volume_opt`, with `thre=n`, in TheDatumOrg/VUS commit `467bace0c103f5c2bdb4900306d82939bcac8ff9`. Tied repeated thresholds add zero area. The target includes dynamic label mass, sum-and-clip overlapping ramps, expanded-interval existence reward, ROC trapezoids, the terminal $(1,1)$ ROC point, and the PR right-endpoint step integral. Other historical VUS routines can use different semantics. We return scalar integrals, not the full surface coordinates.

2 Exact target and sparse events

Let $y_i \in \{0,1\}$ contain both classes and let s_i be a finite score, for $i=1,\dots,n$. Larger scores indicate greater anomaly evidence. Write $P_0=\sum_i y_i$ and let $w\in\{0,\dots,L\}$ be a tolerance width. The official continuous label $a_i^{(w)}$ equals one on original anomaly intervals. Each interval contributes $\sqrt{1-d/w}$ outside its endpoints for integer distance $1\le d\le\lfloor w/2\rfloor$; contributions are summed and capped at one. For $w=0$ no ramp is added.

At threshold t, define $p_i(t) = \mathbf{1}\{s_i \geq t\}$ and

$$C(t) = \sum_i p_i(t), \qquad \mathrm{TP}(t) = \sum_i a_i^{(w)} p_i(t), \tag{1}$$

$$B(t) = \sum_i (1 - y_i) a_i^{(w)} p_i(t), \qquad P(t) = P_0 + B(t)/2, \quad N(t) = n - P(t). \tag{2}$$

The identity for $P(t)$ follows by resetting original positive labels to one after thresholding the buffer labels, then averaging their mass with P_0. Let K_w be the number of expanded intervals and $H(t)$ the number containing a predicted positive. Overlapping expansions merge; adjacent nonoverlapping expansions remain separate, matching the reference. The curve coordinates are

$$\mathrm{TPR}(t) = \min\{\mathrm{TP}(t)/P(t), 1\}\frac{H(t)}{K_w}, \qquad \mathrm{FPR}(t) = \frac{C(t) - \mathrm{TP}(t)}{N(t)}, \qquad \mathrm{Prec}(t) = \frac{\mathrm{TP}(t)}{C(t)}. \tag{3}$$

Here $N(t) > 0$ because $P(t) \leq (n + P_0)/2 < n$. ROC uses trapezoids from the origin through all distinct descending thresholds to the explicit terminal $(1, 1)$. PR is $\sum_j (\mathrm{TPR}_j - \mathrm{TPR}_{j-1})\mathrm{Prec}_j$, with initial recall zero. VUS is the arithmetic mean of these areas over $w = 0, \ldots, L$.

Let S_L be the union of all positions in maximum-width expanded intervals, $Q = |S_L|$, and let $G \leq Q$ be the number of distinct scores in S_L. The distinction matters: Q counts positions, not distinct thresholds. All nonzero weights and all positions capable of changing existence reward lie in S_L, for every evaluated width.

Lemma 1 (Negative-only runs)**.** *Between consecutive distinct support scores, the states* TP*,* B*,* P*,* N*,* H*, and* TPR *are constant. Intermediate thresholds contribute only a horizontal ROC rectangle and zero PR area.*

Proof. Every newly admitted point outside S_L has zero weight and belongs to no expanded interval at width $w \leq L$. Thus only C changes. Since $\mathrm{FPR} = (C - \mathrm{TP})/N$ with fixed TP, N, all intermediate ROC points have the same height. Trapezoid widths telescope to the endpoint difference. Every PR recall increment is zero. This includes arbitrary ties among off-support scores. □

For a descending support event j at score v_j, compute $c_j^{>} = |\{i : s_i > v_j\}|$ and $c_j^{\geq} = |\{i : s_i \geq v_j\}|$. Write $T = \mathrm{TP}$, $R = \mathrm{TPR}$, and $F = \mathrm{FPR}$ for compact event notation. Suppose the preceding event leaves state (T_o, N_o, R_o, c_o) and the new event has coordinates $(F_j, R_j, \mathrm{Prec}_j)$. Its exact contribution is

$$f_j^{-} = (c_j^{>} - T_o)/N_o, \tag{4}$$

$$A_j^{\mathrm{ROC}} = (c_j^{>} - c_o)R_o/N_o + (F_j - f_j^{-})(R_j + R_o)/2, \tag{5}$$

$$A_j^{\mathrm{PR}} = (R_j - R_o)\mathrm{Prec}_j. \tag{6}$$

The initial state is $(0, n - P_0, 0, 0)$; thereafter $c_o = c_{j-1}^{\geq}$. The first term is the negative rectangle; the second admits the whole tied support event, including off-support points tied with it. Omitting the rectangle and connecting retained event endpoints directly is generally wrong. After the last event, the remaining negative tail and terminal ROC transition contribute

$$\frac{(n - c_G^{\geq})R_G}{N_G} + \left(1 - \frac{n - T_G}{N_G}\right)\frac{1 + R_G}{2}. \tag{7}$$

For example, $y = (1, 0, 1, 0)$, $s = (3, 2, 1, 0)$, and $L = 0$ give exact ROC area 0.625. Connecting only support endpoints incorrectly gives 0.8125. The exact step-PR area is 0.75.

Lemma 2 (Two interval distances)**.** *For a non-original position, the official buffer weight is determined by its two nearest distinct anomaly intervals.*

Proof. Each active ramp contributes at least $\sqrt{1/2}$. If at least two intervals contribute, their sum exceeds one and clipping returns one. Otherwise the unique contribution is determined by the nearest interval. Hence the weight is zero when $d_1 > \lfloor w/2 \rfloor$, one when $d_2 \leq \lfloor w/2 \rfloor$, and $\sqrt{1 - d_1/w}$ otherwise. Original positive positions remain one. The nearest two distances are computed from the two neighboring intervals on either side in sorted interval order. □

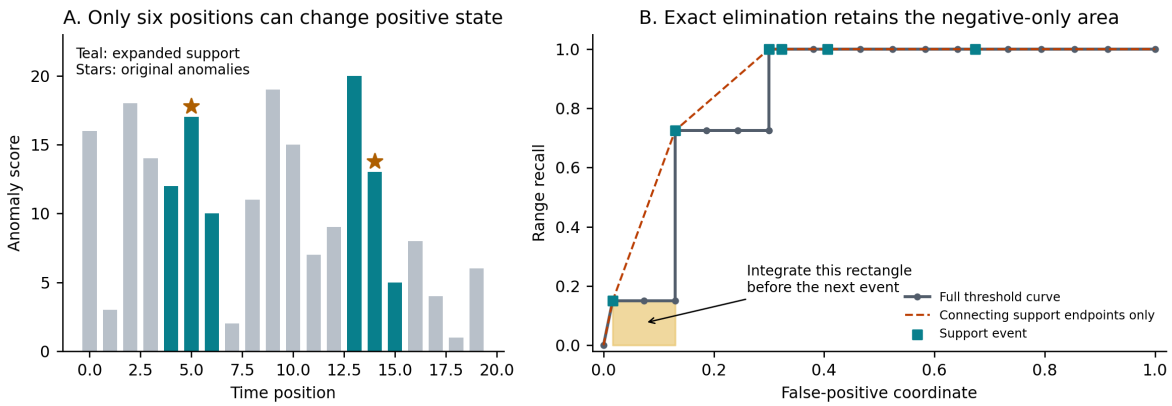


Figure 1: A width-two example. Negative-only thresholds move horizontally along the ROC curve. Integrating these rectangles and the subsequent tied support transitions preserves the full-threshold area (0.841642); joining support endpoints alone gives 0.897580.

Theorem 1 (Support-sensitive exact evaluation)**.** *The two pinned scalar VUS integrals can be computed in* $O(n\log(Q+1)+(L+1)Q)$ *time and* $O(n+Q)$ *memory in a unit-cost real-arithmetic model with score comparisons and square roots. The returned values equal the reference integrals in exact arithmetic.*

Proof. Extract intervals and S_L in $O(n+Q)$ time. Sort the Q support scores and group their G distinct values. Binary-search each of the n scores among these values and accumulate counts to obtain every $c_j^{>}, c_j^{\geq}$, without sorting off-support scores. This costs $O(Q\log Q + n\log(G+1))$, bounded by $O(n\log(Q+1))$ because $1 \leq G \leq Q \leq n$. Precomputing the two nearest interval distances costs $O(Q\log(K+1))$, absorbed by the same bound since the number of original intervals $K \leq Q$.

For each width, construct weights in $O(Q)$ using Lemma 2. Assign disjoint expanded interval memberships in $O(Q+K)$ and find each interval's earliest support-score group. Grouped weighted sums and cumulative sums yield TP, B, and H for all G events in $O(Q+G+K) = O(Q)$ time. Lemma 1 and the event formulas reproduce every omitted threshold contribution, including the terminal tail. Average over $L+1$ widths. Arrays for the input, a position map, support positions, grouped states, and interval metadata require $O(n+Q)$ memory. Floating-point implementations approximate this exact-arithmetic identity; their numerical error is measured separately. □

The compiled scalar kernel streams the width average. The NumPy diagnostic implementation retains per-width areas and uses an additional $O(L)$ space. For dense support, $Q = \Theta(n)$, no asymptotic improvement over a dense sweep is promised. The analysis establishes an upper bound.

3 Prior art and implementation comparisons

Boniol et al. introduce optimized VUS implementations with static/dynamic label splitting, maximum-buffer support restriction, and memory tradeoffs [2]; those are substantive prior optimizations. Sylligardos reports preliminary VUS speedups up to two orders on CPU and three on GPU over approximately 1,900 TSB-UAD series [3]. That workshop paper does not disclose the detailed algorithm, and a corresponding released implementation was not located in our 19 October 2026 search. Its mechanism could overlap with ours. We therefore do not claim the first fast VUS implementation, the first 100-fold acceleration, or demonstrated superiority over that method.

The released Go `adscore` package implements an incremental dense rank sweep [4]. Native execution of its unmodified v0.0.23 package confirms different semantics: static positive mass, original-interval existence, maximum-combined overlapping ramps, and trapezoidal PR. On $y = (0,1,0,0,1,0)$, $s = (1,2,3,4,5,6)$, and $L = 4$, its ROC/PR values are $(0.635984, 0.638587)$, versus $(0.689265, 0.683409)$ for the pinned target. It is prior art for rank sweeps, but not an interchangeable exact-target runtime baseline.

Current dtaianomaly commit `557b4357` provides a compiled, parallel support-scan implementation with the target semantics [5]. We execute its original method bodies and compiled kernel unchanged through a dependency-isolating adapter, obtain both integrals from one curve calculation, and use one thread. Its support restriction differs from eliminating entire negative-only threshold runs and aggregating all support events in one pass. Both default 250-threshold and full-threshold variants are evaluated.

4 Experimental protocol

The protocol, exact source hashes, score archive selection, and a pre-outcome addendum are included with the artifact. The dtaianomaly comparator was selected before execution; a separate, retrospectively written addendum records that decision and its timing. One thousand independently seeded random cases vary length, label density, ties, constant scores, and buffer width; six structured cases cover edges and overlapping/adjacent expansions. Separately authored validation scripts add 1,960 exhaustive tiny cases and 120 new random cases, including extreme finite scores and arbitrary boundary labels.

We use three released score archives from Zhu et al. [6]: Chronos-Bolt base, Time-MoE 200M, and TimesFM 1.0 200M, each at context length 64. Models were chosen by a frozen family/context/lexicographic rule. All common valid series from the 350-series TSB-AD-U evaluation split [7] are included; no model is trained or queried. Scores are read unchanged. Their source README documents leading zero padding for Chronos-Bolt and Time-MoE; we do not reproduce that paper's corrected model leaderboard. Original dataset licenses vary, and raw data and scores are not redistributed.

The main experiment evaluates maximum widths 10 and 30. Each timing includes validation, preprocessing, sorting or histogramming, and integration. Three repetitions use a seeded randomized method order; JIT compilation/first-call time and downloads are excluded and recorded separately. The machine is an Apple M1 Pro, Python 3.12.4, with all compute libraries restricted to one thread. We compare NumPy sparse, Numba sparse, NumPy dense, Numba dense with precomputed nearest distances, and a separately implemented Numba dense method constructing every original sum-and-clip ramp directly. The principal speed ratio uses the fastest median among all three dense implementations for each input, favoring the dense baselines.

The final timed output for each full trace is cross-checked between all five implementations. Official full-threshold and sampled-threshold equivalence are checked on fixed 256-position crops around each first anomaly, when both classes occur. Published external runtimes are measured on the prespecified first 12 common series (all NAB, lengths 1,000–10,319), with all three score vectors and both widths. Official 250-threshold timing has one repetition; dtaianomaly has three. The five in-repository methods use randomized order. External methods use the fixed order official-250, dtaianomaly-250, then dtaianomaly-exact. This external timing subset is not representative of all dataset families.

Two supplementary protocols were fixed after review of the main setup and before their respective executions. A mechanism ablation replaces only the score-count preprocessing with a global full-score sort, then calls the identical compiled sparse integration kernel. It crosses two lengths $(10^4, 10^5)$, four positive densities $(0.001, 0.01, 0.1, 0.5)$, four interval counts $(1, 10, 100, 1000)$, and four maximum widths $(0, 10, 30, 100)$. Sixteen geometrically infeasible cells are excluded before execution, leaving 112 cases. Five implementations run in randomized order for three repetitions. The second supplement tests the actual published implementation across source families; its selection and censoring rules are reported with the results.

For sampling effects, all three unordered model pairs per series/width are examined. A reversal requires opposite signs and absolute metric differences above 10^{-10} in both calculations; ties are reported separately. Series, not score vectors or width/model repetitions, are the units for uncertainty summaries. These are changes in ordering induced by metric computation, not claims that one model is superior.

5 Results

The completed experiment contains 350 series from 23 source families, 1050 released score vectors, and 2100 score/width evaluations. Sequence lengths range from 1,000 to 900,000. The independent final artifact gate passes all coverage, finite-value, source-hash, internal, crop, and external-comparator checks. The maximum absolute numerical discrepancy is 3.69×10^{-13}.

The median compiled-sparse speedup over the fastest dense baseline is 2.06×, with geometric mean 2.02×. The 95% series-cluster bootstrap interval for the geometric mean is [1.93, 2.12]; resampling whole source families

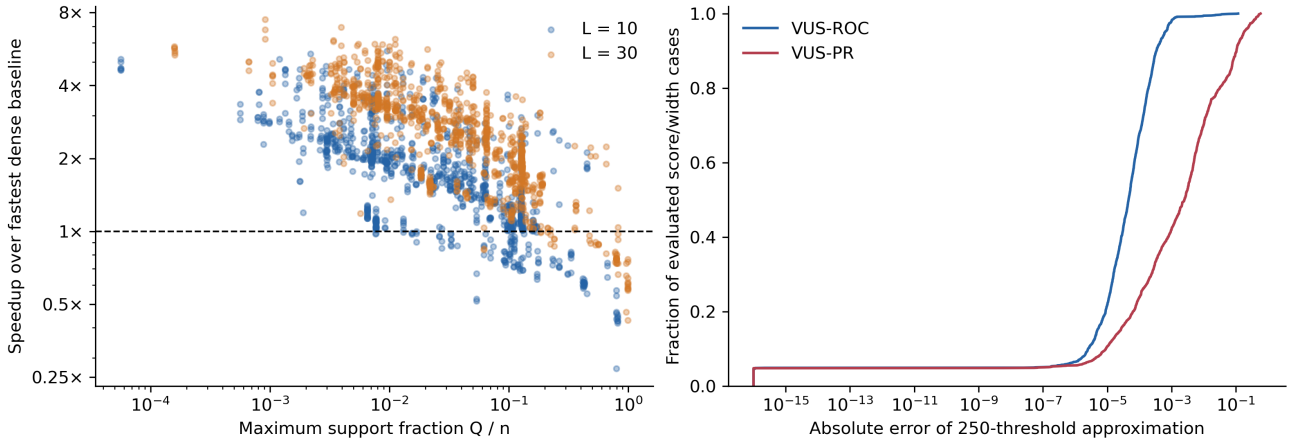


Figure 2: Left: all 2100 paired runtime comparisons, using the fastest of three dense implementations per input. Right: threshold-sampling error on the same fixed score vectors. Zero errors are displayed at 10^{-16} for the logarithmic axis.

gives [1.56, 2.48]. These are descriptive uncertainty summaries for the fixed corpus. Sparse evaluation is faster in 1916 of 2100 cases, while 184 cases are slower or equal. The full speedup range is 0.27–7.50×. Figure 2 retains these slowdowns rather than filtering dense support.

Metric	Median error	95th percentile	Maximum error	Reversals / pairs
VUS-ROC	4.59×10^{-5}	6.21×10^{-4}	0.117	5 / 2100
VUS-PR	2.50×10^{-3}	0.178	0.583	84 / 2100

Table 1: Errors are absolute differences between the same metric at 250 sampled thresholds and all thresholds. Reversals require strict opposite order in both calculations.

For VUS-ROC, strict reversals occur on 2 unique series across 2 families, and the unique top model changes in 2 of 700 series/width cases. Exact and sampled comparisons contain 96 and 95 ties, respectively.

For VUS-PR, strict reversals occur on 55 unique series across 13 families, and the unique top model changes in 50 of 700 series/width cases. Exact and sampled comparisons contain 96 and 95 ties, respectively.

The largest observed PR discrepancy occurs on the 195,000-point UCR HumanActivity series with Time-MoE scores and $L = 30$: exact VUS-PR is 0.666890, versus 0.084159 at 250 thresholds. The scores are unchanged between calculations.

On the bounded 12-series NAB external comparison, the median speed ratios relative to compiled sparse evaluation are 53.86× for full-threshold dtaianomaly, 8.92× for 250-threshold dtaianomaly, and 149.29× for official 250-threshold evaluation. The latter two compare exact computation with a coarser approximation. These results do not establish the same advantage on other families or against the unavailable accelerated VUS implementation.

In the frozen fixed-support scaling experiment ($Q = 102$, $L = 30$), the one-million-point case takes 0.0626 seconds for compiled sparse evaluation, an 8.92× speedup over the fastest dense implementation, with five randomized-order repetitions.

5.1 Mechanism and the density/width tradeoff

On the 112-case synthetic grid, replacing full-score sorting with support-score counting gives a geometric-mean end-to-end speedup of 1.11× with the same integration kernel, range 0.94–1.80×. The adapter deliberately holds support grouping and all other preparation fixed, isolating the score-counting choice. Relative to the fastest dense implementation, the complete sparse method has geometric-mean speedup 1.37×, range 0.26–7.53×. There are 69 cases with at least a 1.1× speed ratio and 34 with ratio at most 1/1.1. The remaining cases lie between those thresholds. Thus the count-preprocessing improvement is modest on this grid; sparse event integration supplies the larger opportunity as support shrinks and the width range grows.

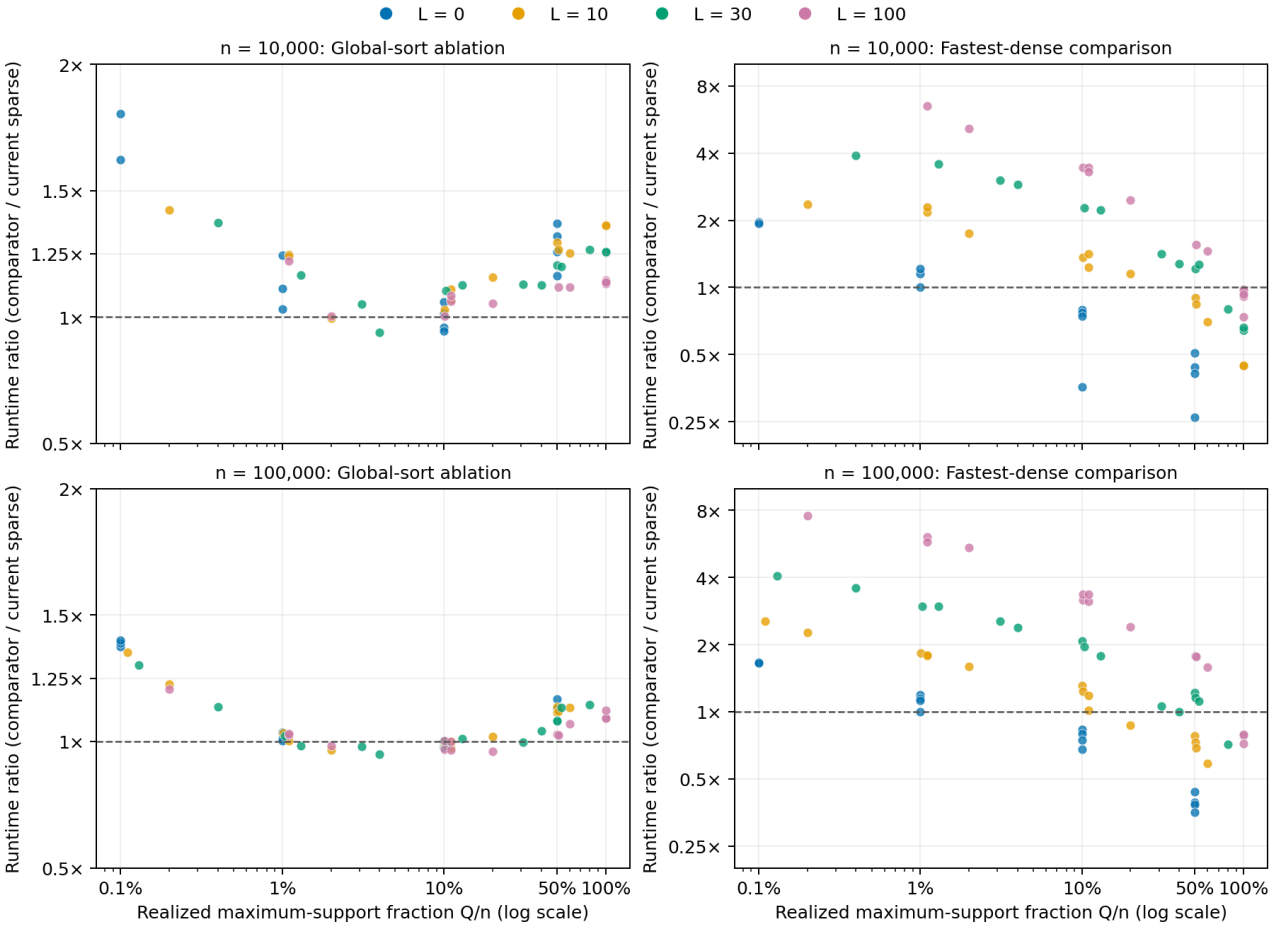


Figure 3: Mechanism ablation: each point is one predeclared synthetic cell, using medians of three randomized repetitions. Support fraction Q/n is measured after expansion. The dense comparator is the fastest of three dense implementations. The full-sort variant keeps support grouping and the compiled sparse integration kernel fixed.

5.2 Published implementation across source families

The post-review supplement selects the shortest and median-length series within each family, deduplicating coincident choices. It contains 42 full-length series from all 23 families, uses Chronos-Bolt base scores and $L = 30$, and preserves all vectors. Methods run in a seeded random order, with three repetitions per method. Each call has a ten-second post-warmup time limit; late or unfinished calls are recorded as right-censored. Compilation and subprocess startup are excluded. Table 2 reports speed ratios only for completed pairs, whose subset differs across threshold modes. No numerical speed ratio is imputed for a censored result.

The maximum absolute discrepancy for returned outputs, compared with the corresponding exact or 250-threshold main result, is 9.55×10^{-15}. All selection, timeout, environment, and per-call receipts are retained.

6 Limitations and reproducibility

The target is a specific, pinned scalar implementation. Exactness does not resolve scientific criticisms of the underlying metric, extend it to all historical variants, or produce the full surface. The acceleration depends on sparse expanded support and can disappear for dense support. Floating-point equivalence is numerical, not bitwise. The NumPy/Numba dense implementations are independently written algorithmic baselines, not published competing systems. The actual released dtaianomaly comparison is bounded, and the Sylligardos fast implementation remains unbenchmarked.

dtaianomaly mode	Completed	Censored	Families represented	Median ratio
250 thresholds	42	0	23	8.17×
All thresholds	35	7	19	143.85×

Table 2: Actual pinned dtaianomaly code on 42 source-stratified full-length inputs. Ratios divide the published baseline's median time by compiled sparse exact evaluation. The 250-threshold row compares an approximation with exact computation. Completion-conditioned medians are not full-corpus runtime estimates. Family counts require at least one completed pair within the family.

The repository includes source, tests, a rigorous derivation, frozen protocols, download and source-checksum receipts, per-input timing and metric records, negative results, and scripts for regenerating figures and this manuscript. All experiments reuse released score vectors on a single CPU; no models are retrained. Experiments were executed on 19 October 2026 UTC. Development and in-session code review used AI assistance; the separately authored checks are distinct test implementations within this session, not an external laboratory reproduction.

References

[1] J. Paparrizos et al. Volume Under the Surface: A New Accuracy Evaluation Measure for Time-Series Anomaly Detection. PVLDB 15(11), 2022. https://www.vldb.org/pvldb/vol15/p2774-paparrizos.pdf.

[2] P. Boniol et al. VUS: Effective and Efficient Accuracy Measures for Time-Series Anomaly Detection. VLDB Journal 34, 2025. https://arxiv.org/abs/2502.13318.

[3] E. Sylligardos. Fast or Accurate? Rethinking Time Series Anomaly Detection. VLDB PhD Workshop, 2025, Section 4. https://www.vldb.org/2025/Workshops/VLDB-Workshops-2025/PhD/PhD25_17.pdf.

[4] P. Stergiotis. boxer/adscore, v0.0.23, 2026. https://pkg.go.dev/github.com/stergiotis/boxer/public/analytics/timeseries/adscore.

[5] ML-KULeuven. dtaianomaly, pinned revision `557b4357`, 13 October 2026. https://github.com/ML-KULeuven/dtaianomaly.

[6] X. Zhu, L. Carpentier, M. Verbeke. When Foundation Models are One-Liners: Limitations and Future Directions for Time Series Anomaly Detection. ICLR 2026. Scores: KU Leuven RDR, V1, https://doi.org/10.48804/MCVI4N.

[7] Q. Liu and J. Paparrizos. The Elephant in the Room: Towards a Reliable Time-Series Anomaly Detection Benchmark. NeurIPS Datasets and Benchmarks, 2024. https://github.com/TheDatumOrg/TSB-AD.

[8] K. Kotowski et al. European Space Agency Benchmark for Anomaly Detection in Satellite Telemetry. 2024, revised 2025. https://arxiv.org/abs/2406.17826.

A A fixed uniform-rank grid can lose almost all PR area

The following proposition concerns the particular uniform-rank sampling rule used by the pinned reference, with all input scores distinct. It does not apply to every adaptive threshold-selection scheme.

Proposition 1. *Fix an integer threshold budget $T \geq 2$, a maximum width $L \geq 0$, and $\varepsilon > 0$. There are finite-score binary-label sequences for which the absolute difference between exact VUS-PR and the reference's T-threshold VUS-PR is greater than $1 - \varepsilon$.*

Proof. Take one anomaly interval of length m, with at least $\lfloor L/2 \rfloor$ positions of padding on both sides. Give its original positives score ranks $2, \ldots, m+1$, put the highest-scored point outside the maximum-width support, and give all buffer positions the next consecutive ranks. Give all other positions distinct lower scores. Append further lower-scored negatives at the right, preserving the existing geometry and scores.

For every $w \le L$, the jth original positive is admitted before any buffer point. Its recall is j/m, precision is $j/(j+1)$, and existence reward is one. Recall reaches one after the last original positive. All subsequent PR increments are zero, so exact VUS-PR is

$$E_m = \frac{1}{m}\sum_{j=1}^{m}\frac{j}{j+1} = 1 - \frac{H_{m+1}-1}{m}, \tag{8}$$

where H_k is the kth harmonic number.

The sampled grid has one-based ranks $1+\lfloor j(n-1)/(T-1)\rfloor$, for $j = 0, \ldots, T-1$. For sufficiently large n, its second rank $k_n = 1+\lfloor (n-1)/(T-1)\rfloor$ contains all maximum-width support. The first sampled recall is zero. Let $A_w = \sum_i a_i^{(w)}$ denote the full label mass. At the second threshold, TP $= A_w$, $B = A_w - m$, and $P = (m+A_w)/2$. Thus clipped recall and existence reward both equal one, and precision is A_w/k_n. No later sampled threshold adds PR area. The sampled VUS-PR therefore equals $\overline{A}/k_n$, where $\overline{A} = (L+1)^{-1}\sum_{w=0}^{L} A_w \le m + L$.

First choose m so that $(H_{m+1}-1)/m < \varepsilon/2$; then append enough negatives that k_n exceeds all support ranks and $2(m+L)/\varepsilon$. The exact area exceeds $1-\varepsilon/2$ and the sampled area is less than $\varepsilon/2$, proving the claim. □

The artifact includes deterministic checks at $T = 250$, $m \in \{1, 10\}$, $L \in \{0, 6\}$, and $n \in \{10^4, 10^5\}$ against the two closed-form values.